ADR 0069 — A conflict-content resolution seeds from the marker file, and a `conflict-v1:` token pins exactly what was served

Date: 2026-08-23

Status: Accepted — design only, no code. Third and final slice of M4.31 (#84), building on ADR 0063 (the read model) and ADR 0064 (whole-side resolution).

Context

ADR 0064 gave conflicts a write path, and drew a line on purpose: decision 3 says `Resolution` "names a side and never carries bytes", because a `Plan` is hashed, reviewed and replayed, and putting arbitrary file content inside one would mean a hash covering bytes nobody reviewed as a plan, an approval that cannot be meaningfully re-verified, and a replay path that could rewrite a file from a stale approval. #430 (ADR 0068) built the last whole-side surface. What remains under #84 is block-level and line-level resolution and safe manual editing — issue #432 — and doing that means the thing ADR 0064 deliberately deferred: the user's chosen result, actual file content, has to reach the server.

An independent read-only review was commissioned before any design commitment (`~/projects/_claude-outputs/2026-08-22_fable-conflict-content-transport.md`, read against `main@d8adc5d6`) to answer one question: does the pattern already

solved for staging (`StageSelection`, hunk-level patches moved via a hashed, generation-pinned plan) transfer to conflicts? Its citations were spot-checked against source before this ADR relied on them — this repo's plan citations have been wrong before, including once against a function that never existed.

Verified directly, not merely cited:

- `GitOperation::ResolveConflict`'s own doc comment (`crates/git-vista-protocol/src/plan.rs:574-579`) states plainly: "Every variant of `Resolution` names a side rather than supplying bytes, so a plan stays small and hashable. Line-level resolution... needs the `patch_plan` machinery and its own decision; it is not smuggled in here." ADR 0064 anticipated this ADR by name.
- The coordinator guard (`crates/git-vista-server/src/planner.rs:365-390`) spans `refuse_if_git_busy → validate → enforce_fresh → pin_recovery → execute` as one lock hold, exactly as claimed. Any content executor's worktree write and index write both land inside it.
- The `diff-v1:` recipe (`crates/git-vista-server/src/handlers/read.rs:1060-1078`) confirms the precedent this ADR follows: `read_generation_inputs` (HEAD, every ref, the index checksum) plus a worktree slot — here, a SHA-256 of the served patch bytes — folded into one digest and namespaced `diff-v1:{generation}`.
- The index-checksum path (`crates/git-vista-git/src/identity.rs:222-229`) confirmed a real gap: "A repository with no index yet... contributes nothing" — **silently**. No `debug_assert`, no tagged unknown state. Worth fixing generally; not specific to this ADR, so not fixed here.

The verification changed nothing in the recommendation. It is recorded because the standing discipline here is "verify against source before it becomes a decision," and this ADR is exactly the

kind of document a later session will build on without re-checking.

The decision that had to come first: what does the editor start from?

Two shapes were on the table.

A — seed the editor from the working-tree marker file, the same `<<<<<<<`/`=======`/`>>>>>>>` bytes `git merge` already wrote to disk.

B — compose the editor's starting text from the three stage panes (`base`/`ours`/`theirs`), independent of whatever git left in the working tree.

Fable's sharpest finding governs this choice: **porcelain v2's unmerged `u` lines carry the three stage OIDs but no worktree hash, and the index checksum does not cover worktree bytes either.** So under (A), the single document the user actually edits is the single input **no existing staleness mechanism can see**. Edit that file with another tool mid-resolution, and nothing before this ADR would notice.

Decided: A, with a new digest that makes the gap visible instead of avoiding it.

Composing from panes (B) would sidestep the blind spot by never showing the user the file git actually wrote — but that is worse, not safer. Every terminal-based merge tool in existence, and this project's own read model (ADR 0063's `Result` pane already shows "what git actually wrote, markers and all"), works from that file. Inventing a different document than the one on disk means the app and the terminal can disagree about what "the conflict" currently looks like, and disagreement about ground truth is a

worse failure than a closeable gap. The gap closes with one mechanism, below; the alternative would have created a permanent, structural mismatch.

Decision

1. A new operation beside `ResolveConflict`, not a field added to it. `Resolution` still names a side and never carries bytes — ADR 0064's decision 3 is unweakened, not amended. Content resolution is its own vocabulary member, the same way `TakeDeletion` is its own variant rather than a reinterpretation of `TakeOurs`-on-an-absent-side (ADR 0064 decision 4).

```
GitOperation::ResolveConflictContent {
    /// Repository-relative path, exactly as the conflict scan repo
    path: WorktreePath,
    /// The stage OID triple the user resolved against. `None` mean
    /// stage was Absent. Re-verified by exact equality inside the
    expected_stages: [Option<CommitOid>; 3],
    /// `conflict-v1:` token of the document actually served: repos
    /// generation (HEAD, refs, index checksum) plus a digest of th
    /// file bytes the editor was seeded with, folded in the same w
    /// `diff-v1:` folds patch bytes.
    expected_source: GenerationToken,
    /// The user's chosen result. Hash-bound by `OperationHash` lik
    /// `StageSelection`'s patch — reviewed, not smuggled.
    content: String,
}
```

2. A new token namespace, `conflict-v1:`, following the `diff-v1:` recipe exactly. Repository slots from `read_generation_inputs` (HEAD, every ref, the index checksum — which already changes with any stage entry, since stage 1/2/3 entries live in the index). Worktree slot: SHA-256 of the marker-

file bytes served, with the path folded in. Handler mints outside the lock when serving the resolver; executor re-mints inside the lock before writing. Two-phase, exactly `StageSelection`'s pattern.

3. The executor re-scans and refuses on any mismatch, inside the guard, before writing anything. In order:

1. the path is still conflicted (and the scan itself succeeded);
2. eligibility holds — `all_sides_readable()` first, then `text_resolvable()`, the same predicate #430's `ResolutionSurface` asks client-side rather than a second copy of the rule;
3. the three live stage OIDs equal `expected_stages` exactly;
4. the re-minted `conflict-v1:` token equals `expected_source`.

Any failure refuses the whole operation before the file is touched.

Amended 2026-08-23, after implementation. This ADR originally listed eligibility **after** the stage-OID check. The executor ships with eligibility second, and that order is the better one: an ineligible path (binary, a deletion, an unreadable side) should be told *why it cannot be resolved as text* rather than *that its stages moved* — the eligibility answer is the more informative refusal, and it is true regardless of whether the stages moved too. Recorded as an amendment rather than silently reordered, because a decision record that quietly matches the code teaches nothing about which was wrong. Both orders refuse before any write; nothing about the safety argument changes.

3a. Two further gates the implementation added, neither in this ADR's first draft. Both came out of the post-implementation adversarial review:

- **A symlink at the conflicted path is refused outright.** `symlink_containment_guard` refuses symlinks that *escape* the worktree and refuses directories — never an in-worktree symlink. That is the dangerous one here: `tokio::fs::write`

follows the link and writes its **target**, while `git add -- <path>` stages the link **object**, so the resolution would land in an unrelated tracked file while the conflicted path staged something else, and the half-state message would be false. `conflicts::scan` cannot see this — it reads the index, never the worktree's file type.
- **The write targets the joined path, not the canonicalised one**, so both legs name the same file by construction rather than by the symlink check alone holding.

3b. The `conflict-v1:` token pins only the first `FILE_CONTENT_CAP` bytes, and this is a stated limit rather than a solved problem. Both the serving handler and the executor's re-mint read the marker file through the same bounded reader (2,000,000 bytes). Two states of a marker file larger than that, identical up to the cap and differing only past it, therefore mint the **same** token, and gate 4 cannot tell them apart. Nothing currently consults the `truncated` flag `ConflictSource` already carries.

Low severity — it needs a >2 MB conflicted text file whose change is confined entirely past the cap — but it is real, and it narrows gate 4's guarantee from "the served document" to "the served document's first 2 MB". Written down because a guarantee overstated in a decision record is how a later reader builds on something that was never true. The fix, if it is ever wanted, is to refuse a content resolution whose source was served truncated.

4. All three stages are checked, not only the ones the outcome nominally depends on. `TakeOurs` only reads stage 2 at execution time and is self-anchoring by construction; a content resolution was decided by looking at all three panes, so "the picture you decided against has changed" is the invariant — not "the bytes you happened to copy have changed". Any of the three moving invalidates the composition and must refuse.

5. Write-then-add is not atomic, and the executor reports the half-state honestly rather than hiding it. `git apply` refuses atomically on drifted context; writing a file and then `git add`-ing it has no equivalent guarantee. If the write succeeds and the add fails, the operation reports exactly that — the same posture the existing whole-side add-leg failure already takes (`planner.rs`, the `ResolveConflict` executor).

6. `RiskLevel` is `Reversible`, with the caveat named in the journal text — never `Safe`. `StageSelection` earns `Safe` because it is index-only: the working tree keeps every edit regardless of outcome. This operation overwrites a worktree file. If the marker file was hand-edited outside the app between being served and being resolved, that edit's only copy is destroyed by the write — `ConflictRecreatableWhileInProgress` recovers the *conflict* (`git checkout -m`), not the overwritten edit. This is stated as a fact of the operation's shape, in the journal, not smoothed over.

7. Excluded from the MCP tool surface, restated explicitly rather than left as inheritance. ADR 0064 decision 7 excludes whole-side resolution because choosing requires having seen the sides. A content variant carrying arbitrary bytes inherits that exclusion *a fortiori* — an agent authoring file content from a tool description has seen even less than one picking a side. Recorded here on the record a second time on purpose.

8. Binary, deletion-only, and unreadable-side conflicts refuse at the gate. `NotTextResolvable` conflicts and any path failing `all_sides_readable` are not eligible for content resolution — they have no text to compose a line-level choice from. #430's `ResolutionSurface::text_resolution_allowed` already computes this; the handler reuses it rather than re-deriving the rule.

9. No anchor-coordinate layer is invented. `StageSelection`'s `HunkRef` anchors defend against indexing bugs under a pinned generation and are explicitly "not a staleness mechanism" in their

own doc comment. Here the wire carries the composed result's full bytes, hash-bound directly — the `expected_stages` triple plays the staleness role; there is nothing for a separate anchor layer to do.

What must exist before this is built, not decided by writing it down

A test proving a stage-entry-only change moves the plan generation existed as a gap at review time, and has since closed. Fable's review (2026-08-22, against `main@d8adc5d6`) found the claim asserted only from git's documented porcelain format and from reading the parser — no test pinned it. PR #433 (`test(#432): pin that a stage move is visible to the freshness gate`) closed that gap the same day and is merged on `main`. Its assertion should still be read before implementation starts here, not assumed from this ADR's prose — but the prerequisite itself is satisfied, not outstanding.

The index-checksum `None` case should get an explicit tagged state, the same posture `Obs::Unknown` already uses elsewhere in the planner, rather than silently omitting the slot. Not blocking for this ADR, but noted so it is not rediscovered as a surprise mid-implementation.

Alternatives considered

A `content: String` field on `Resolution` itself. Rejected — this is the exact move issue #432 names as "the thing not to do without argument," for the reasons ADR 0064 decision 3 already gives.

Compose the editor from the three panes (option B above). Rejected: closes the staleness gap by never showing the user the file git actually wrote, producing a document that can

disagree with the terminal about what the conflict looks like. See "The decision that had to come first," above.

An atomic backstop analogous to `git apply`'s context matching. Not available: there is no comparable primitive for "write this file, but only if nothing nearby has drifted." Compensated by the OID-triple check being a *harder* guarantee than context matching ever was — exact identity rather than approximate — and by reporting the write/add half-state honestly instead of pretending atomicity that does not exist.

Reusing `patch_plan`'s `HunkRef` anchors for block/line choices. Deferred, not rejected — see decision 9. If a future line-level UI expresses choices as coordinates into the served panes rather than submitting composed bytes, this ADR's `content: String` shape would need revisiting. That is a UI-shape question for whoever builds #432's actual editor, not a transport question this ADR needs to pre-answer.

Consequences

- `OperationHash`'s guarantees are unweakened: the hash still covers everything reviewed, including the newly-hashable `content` field — nothing bypasses it.
- A resolution applied against drifted stages, a drifted served document, or an ineligible path is refused before any write, by construction of the guard order in decision 3.
- **Gate 4's refusal names no cause, deliberately.** The first implementation said the file "was edited elsewhere while you were resolving it". That is a claim the code cannot make: `conflict_source_token` folds the marker bytes *and* the whole repository generation (HEAD, every ref, the index checksum) into one digest, and `GenerationInputs::generation()` hashes those fields together, so no per-field attribution survives a mismatch. Worse, gate 3 has by then already proven this path's

own stage OIDs unchanged — making "someone edited your file" the *least* likely remaining cause, behind an unrelated branch moving, a fetch landing, or a different file being staged. The sentence now states only what was observed: the repository changed. Caught by the post-implementation honesty review, and it is the same defect class ADR 0063 exists to prevent.

- The operation is `Reversible`, never `Safe`, and the journal says why — matching this codebase's standing rule that a green result must show real evidence, not an optimistic label.
- Block-level and line-level UI, and safe manual editing, can now be built against a settled transport contract. This ADR does not design that UI.
- Criterion "mutation-proven two ways" from #432's acceptance list applies to the executor's guard order once built — at minimum: one mutation removing the OID-triple check, one removing the `conflict-v1:` re-mint, each expected to fail differently.

Signed: max · 2026-08-23T03:35:00-04:00